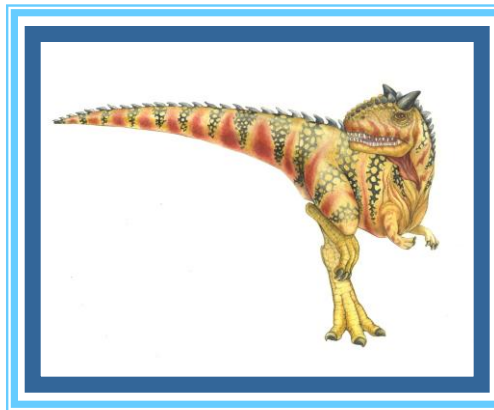
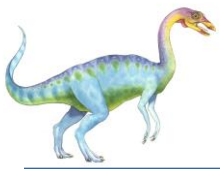


Chapter 5: CPU Scheduling





Chapter 5: CPU Scheduling

- Basic Concepts
- Scheduling Criteria
- Scheduling Algorithms
- Thread Scheduling





CPU Scheduling

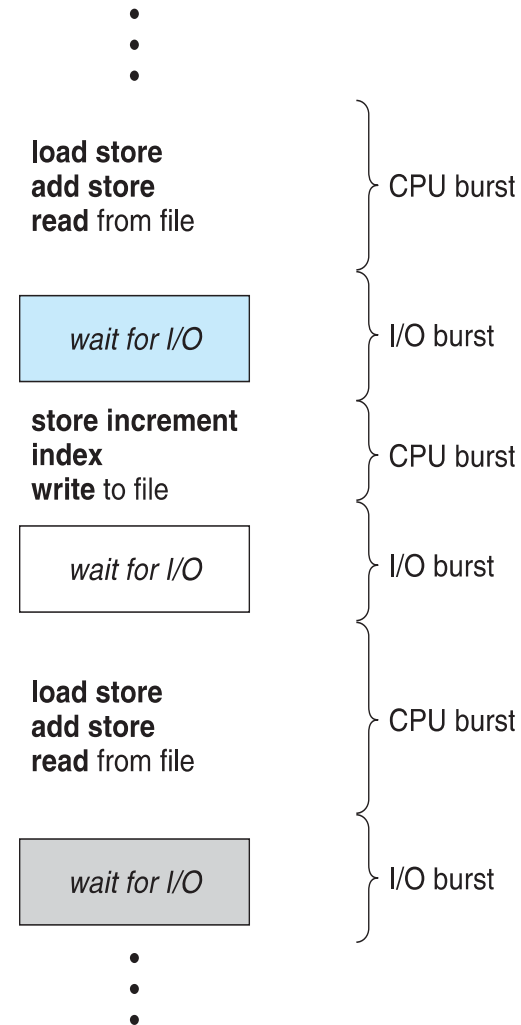
- ❑ basis of multiprogrammed operating systems
- ❑ By switching the CPU among processes, the operating system can make the computer more productive.
- ❑ On operating systems that support threads, it is kernel-level threads—not processes—that are being scheduled by the operating system.
- ❑ the terms **process scheduling** and **thread scheduling** are often used interchangeably





Basic Concepts

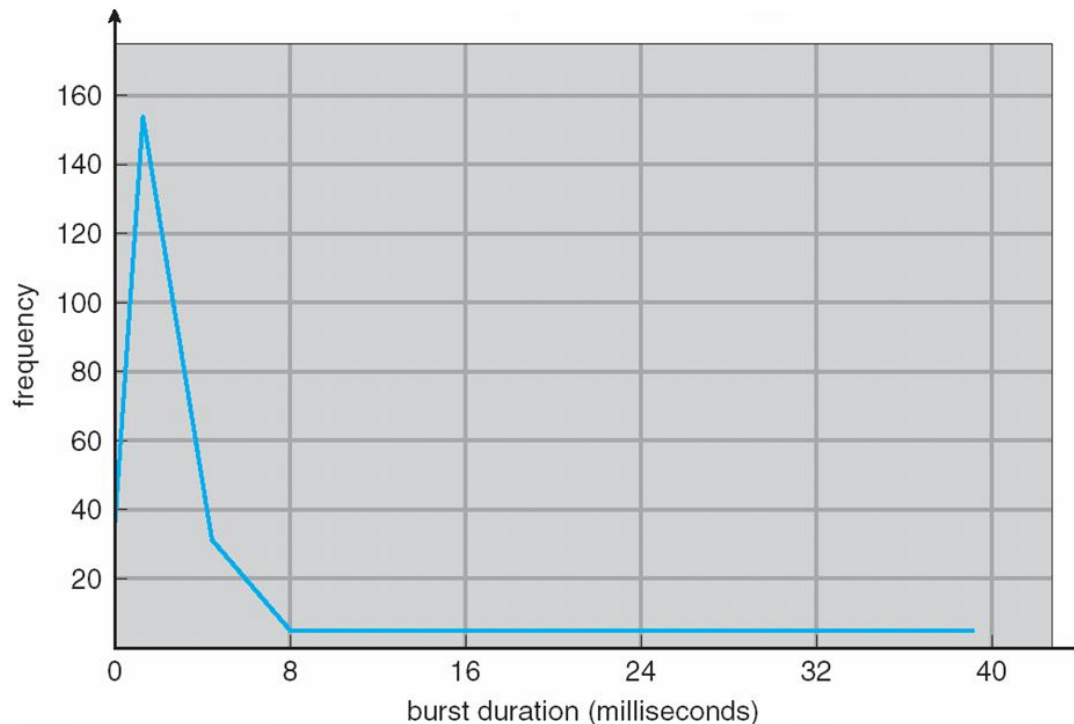
- ❑ Maximum CPU utilization obtained with multiprogramming
- ❑ CPU–I/O Burst Cycle – Process execution consists of a **cycle** of CPU execution and I/O wait
- ❑ Process execution begins with **CPU burst** followed by **I/O burst** which is followed by another CPU burst, then another I/O burst, and so on.





Histogram of CPU-burst Times

- large number of short CPU bursts and a small number of long CPU bursts
- I/O-bound program typically has many short CPU bursts
- CPU Bound program might have a few long CPU bursts
- CPU burst distribution is important in the selection of an appropriate CPU-scheduling algorithm





CPU Scheduler

- ❑ **Short-term scheduler** selects from among the processes in ready queue, and allocates the CPU to one of them
 - ❑ Queue may be ordered in various ways
- ❑ CPU scheduling decisions may take place when a process:
 - ❑ 1. Switches from running to waiting state(as the result of an I/O request or an invocation of wait for the termination of one of the child processes)
 - ❑ 2. Switches from running to ready state (when an interrupt occurs)
 - ❑ 3. Switches from waiting to ready (Completion of I/O)
 - ❑ 4. Terminates
- ❑ Scheduling under 1 and 4 is **nonpreemptive or cooperative**
- ❑ All other scheduling is **preemptive**
 - ❑ Consider access to shared data
 - ❑ Consider preemption while in kernel mode
 - ❑ Consider interrupts occurring during crucial OS activities





Nonpreemptive or cooperative scheduling

- once the CPU has been allocated to a process, the process keeps the CPU until it releases the CPU either by terminating or by switching to the waiting state
- Cooperative scheduling is the only method that can be used on certain hardware platforms, because it does not require the special hardware (for example, a timer) needed for preemptive scheduling.





Preemptive scheduling

- preemptive scheduling incurs a cost associated with access to shared data
 - Consider access to shared data -While one is updating the data, it is preempted so that the second process can run. The second process then tries to read the data, but they are in an inconsistent state
 - affects the design of the operating-system kernel.
 - During the processing of a system call, the kernel may be busy with an activity on behalf of a process
 - Such activities may involve changing important kernel data
 - deal with this problem by waiting either for a system call to complete
 - Consider interrupts occurring during crucial OS activities
 - the sections of code affected by interrupts must be guarded from simultaneous use.
 - The operating system needs to accept interrupts at almost all times; otherwise, input might be lost or output overwritten. So that these sections of code are not accessed concurrently by several processes, they disable interrupts at entry and reenale interrupts at exit





Dispatcher

- ❑ Dispatcher module gives control of the CPU to the process selected by the short-term scheduler; this involves:
 - ❑ switching context
 - ❑ switching to user mode
 - ❑ jumping to the proper location in the user program to restart that program
- ❑ **Dispatch latency** – time it takes for the dispatcher to stop one process and start another running



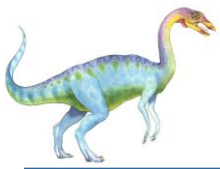


Scheduling Criteria

- ❑ **CPU utilization** – keep the CPU as busy as possible
- ❑ **Throughput** – number of processes that are completed per time unit
- ❑ **Turnaround time** – The interval from the time of submission of a process to the time of completion is the turnaround time
 - ❑ Turnaround time is the sum of the periods spent waiting to get into memory, waiting in the ready queue, executing on the CPU, and doing I/O.
 - ❑ amount of time to execute a particular process
- ❑ **Waiting time** – the sum of the periods spent waiting in the ready queue.
- ❑ **Response time** – amount of time it takes from when a request was submitted until the first response is produced, not output (for time-sharing environment)

- ❑ Therefore desirable to **maximize CPU utilization and throughput** and to **minimize turnaround time, waiting time, and response time**





Scheduling Algorithm Optimization Criteria

- ❑ Max CPU utilization
- ❑ Max throughput
- ❑ Min turnaround time
- ❑ Min waiting time
- ❑ Min response time





Scheduling Algorithms

First- Come, First-Served (FCFS) Scheduling

- the process that requests the CPU first is allocated the CPU first.
- managed with a FIFO queue.
- When a process enters the ready queue, its PCB is linked onto the tail of the queue.
- When the CPU is free, it is allocated to the process at the head of the queue.
- The running process is then removed from the queue





First- Come, First-Served (FCFS) Scheduling Example

<u>Process</u>	<u>Burst Time</u>
P_1	24
P_2	3
P_3	3

- Suppose that the processes arrive in the order: P_1 , P_2 , P_3
The Gantt Chart for the schedule is:



- Waiting time for $P_1 = 0$; $P_2 = 24$; $P_3 = 27$
- Average waiting time: $(0 + 24 + 27)/3 = 17$





FCFS Scheduling (Cont.)

Suppose that the processes arrive in the order:

$$P_2, P_3, P_1$$

- The Gantt chart for the schedule is:



- Waiting time for $P_1 = 6$; $P_2 = 0$; $P_3 = 3$
- Average waiting time: $(6 + 0 + 3)/3 = 3$
- Much better than previous case
- **Convoy effect** - short process behind long process
 - Consider one CPU-bound and many I/O-bound processes

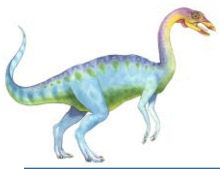




FCFS Scheduling (Cont.)

- FCFS scheduling algorithm is nonpreemptive.
 - Once the CPU has been allocated to a process, that process keeps the CPU until it releases the CPU, either by terminating or by requesting I/O.
- The FCFS algorithm is thus particularly troublesome for time-sharing systems, where it is important that each user get a share of the CPU at regular intervals.





Shortest-Job-First (SJF) Scheduling

- Associate with each process the length of its next CPU burst
 - Use these lengths to schedule the process with the shortest time
- shortest-next-CPU-burst algorithm
- SJF is optimal – gives minimum average waiting time for a given set of processes
 - The difficulty is knowing the length of the next CPU request
 - long-term (job) scheduling in a batch system -Could ask the user
- cannot be implemented at the level of short-term CPU scheduling.
- With short-term scheduling, there is no way to know the length of the next CPU burst

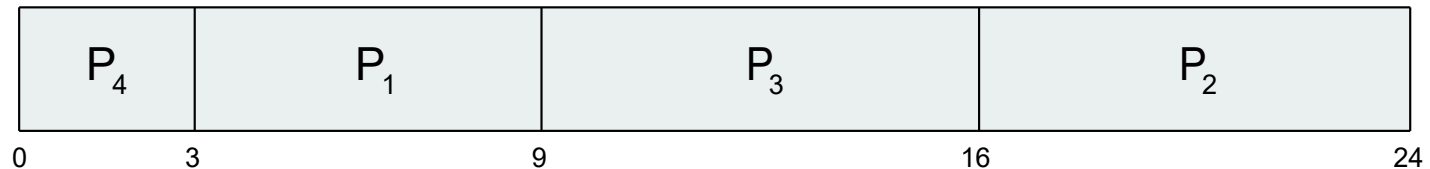




Example of SJF

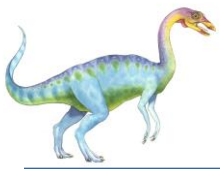
<u>Process</u>	<u>Burst Time</u>
P_1	6
P_2	8
P_3	7
P_4	3

□ SJF scheduling chart



□ Average waiting time = $(3 + 16 + 9 + 0) / 4 = 7$





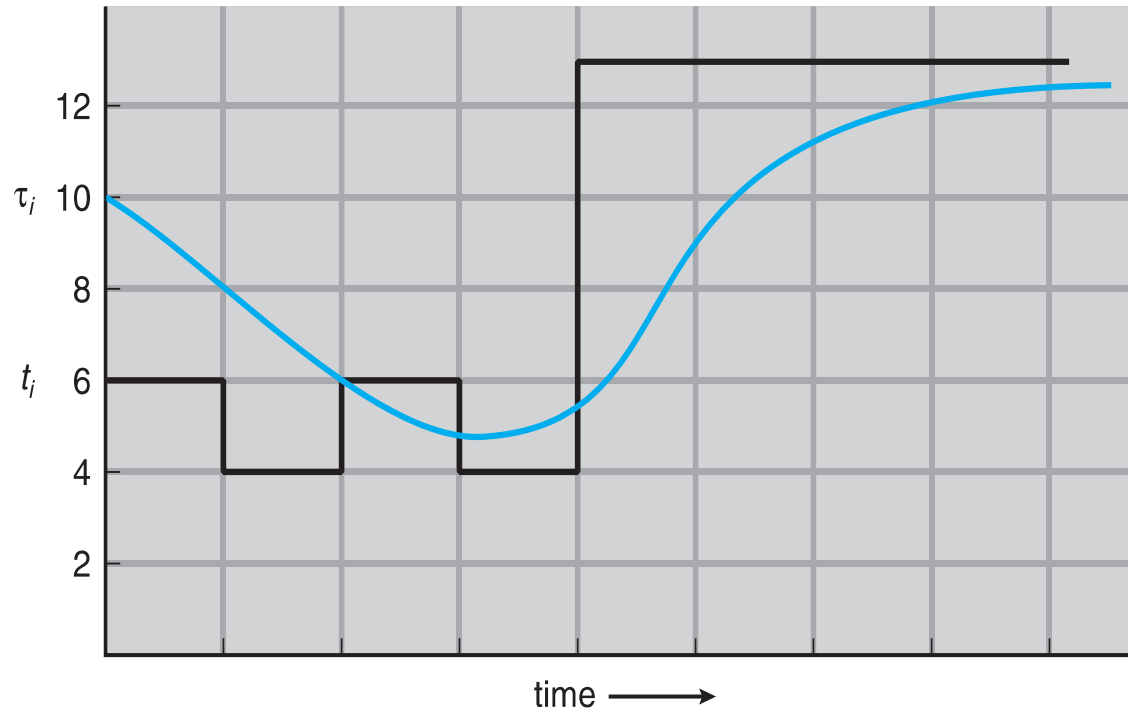
Determining Length of Next CPU Burst

- Can only estimate the length
- Then pick process with shortest predicted next CPU burst
- Can be done by using the length of previous CPU bursts, using exponential averaging
 1. t_n = actual length of n^{th} CPU burst
 2. τ_{n+1} = predicted value for the next CPU burst
 3. $\alpha, 0 \leq \alpha \leq 1$
 4. Define : $\tau_{n+1} = \alpha t_n + (1 - \alpha)\tau_n$.
- α controls the relative weight of recent and past history in our prediction
- Commonly, α set to $\frac{1}{2}$ so recent history and past history are equally weighted.





Prediction of the Length of the Next CPU Burst



CPU burst (t_i)	6	4	6	4	13	13	13	...
"guess" (τ_i)	10	8	6	6	9	11	12	...





Examples of Exponential Averaging

- $\alpha = 0$
 - $\tau_{n+1} = \tau_n$
 - Recent history does not count
- $\alpha = 1$
 - $\tau_{n+1} = \alpha t_n$
 - Only the actual last CPU burst counts
- If we expand the formula, we get:

$$\begin{aligned}\tau_{n+1} = & \alpha t_n + (1 - \alpha)\alpha t_{n-1} + \dots \\ & + (1 - \alpha)^j \alpha t_{n-j} + \dots \\ & + (1 - \alpha)^{n+1} \tau_0\end{aligned}$$

- Since both α and $(1 - \alpha)$ are less than or equal to 1, each successive term has less weight than its predecessor



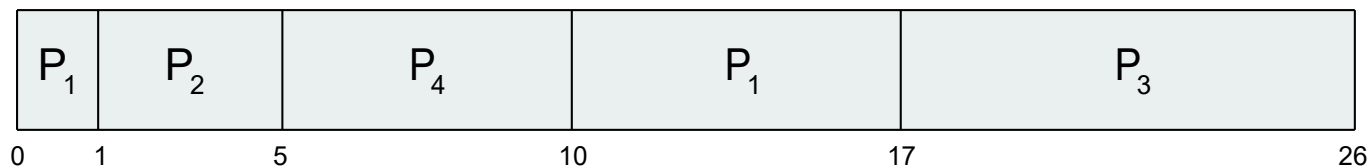


Shortest-Remaining-Time-first Example

- Preemptive version called **shortest-remaining-time-first**
- Now we add the concepts of varying arrival times and preemption to the analysis

<u>Process</u>	<u>Arrival Time</u>	<u>Burst Time</u>
P_1	0	8
P_2	1	4
P_3	2	9
P_4	3	5

- *Preemptive SJF Gantt Chart*



- Average waiting time = $[(10-1)+(1-1)+(17-2)+5-3]/4 = 26/4 = 6.5$ msec





Priority Scheduling

- A priority number (integer) is associated with each process
- The CPU is allocated to the process with the highest priority (smallest integer $\equiv$ highest priority)
 - Preemptive
 - Nonpreemptive
- SJF is priority scheduling where priority is the inverse of predicted next CPU burst time
- Problem $\equiv$ **Starvation** – low priority processes may never execute
- Solution $\equiv$ **Aging** – as time progresses increase the priority of the process

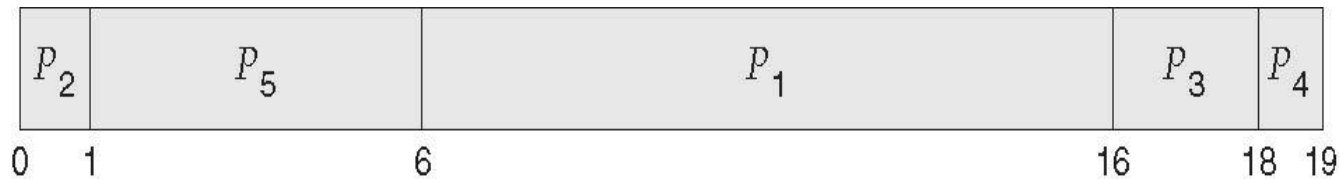




Example of Priority Scheduling

<u>Process</u>	<u>Burst Time</u>	<u>Priority</u>
P_1	10	3
P_2	1	1
P_3	2	4
P_4	1	5
P_5	5	2

□ Priority scheduling Gantt Chart



□ Average waiting time = 8.2 msec





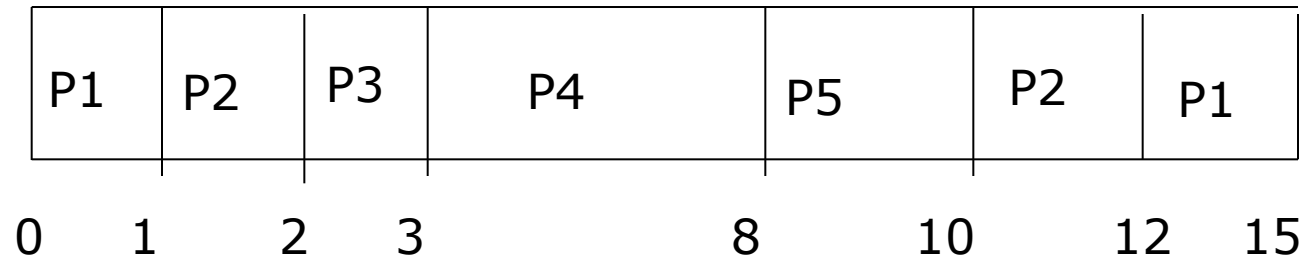
Preemptive Priority Scheduling

Process Id	Arrival time	Burst time	Priority
P1	0	4	4
P2	1	3	3
P3	2	1	2
P4	3	5	1
P5	4	2	1





□ Solution



□ Turnaround time:

$$P1 = 15$$

$$P2 = 12 - 1 = 11$$

$$P3 = 3 - 2 = 1$$

$$P4 = 8 - 3 = 5$$

$$P5 = 10 - 4 = 6$$

Waiting Time:

$$P1 = 11$$

$$P2 = 8$$

$$P3 = 0$$

$$P4 = 0$$

$$P5 = 4$$





Round Robin (RR)

- Each process gets a small unit of CPU time (**time quantum q**), usually 10-100 milliseconds. After this time has elapsed, the process is preempted and added to the end of the ready queue.
- If there are n processes in the ready queue and the time quantum is q , then each process gets $1/n$ of the CPU time in chunks of at most q time units at once. No process waits more than $(n-1)q$ time units.
- Timer interrupts every quantum to schedule next process
- Performance
 - q large $\Rightarrow$ FIFO
 - q small $\Rightarrow q$ must be large with respect to context switch, otherwise overhead is too high

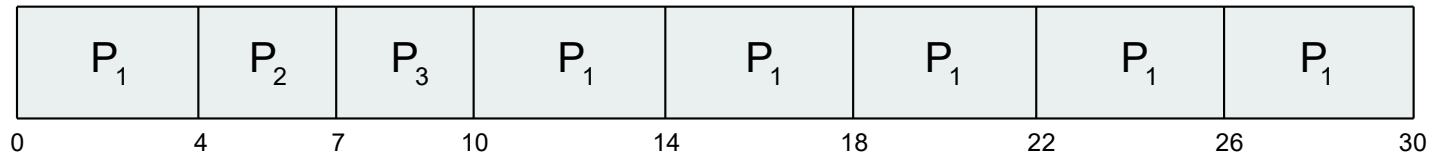




Example of RR with Time Quantum = 4

<u>Process</u>	<u>Burst Time</u>
P_1	24
P_2	3
P_3	3

□ The Gantt chart is:

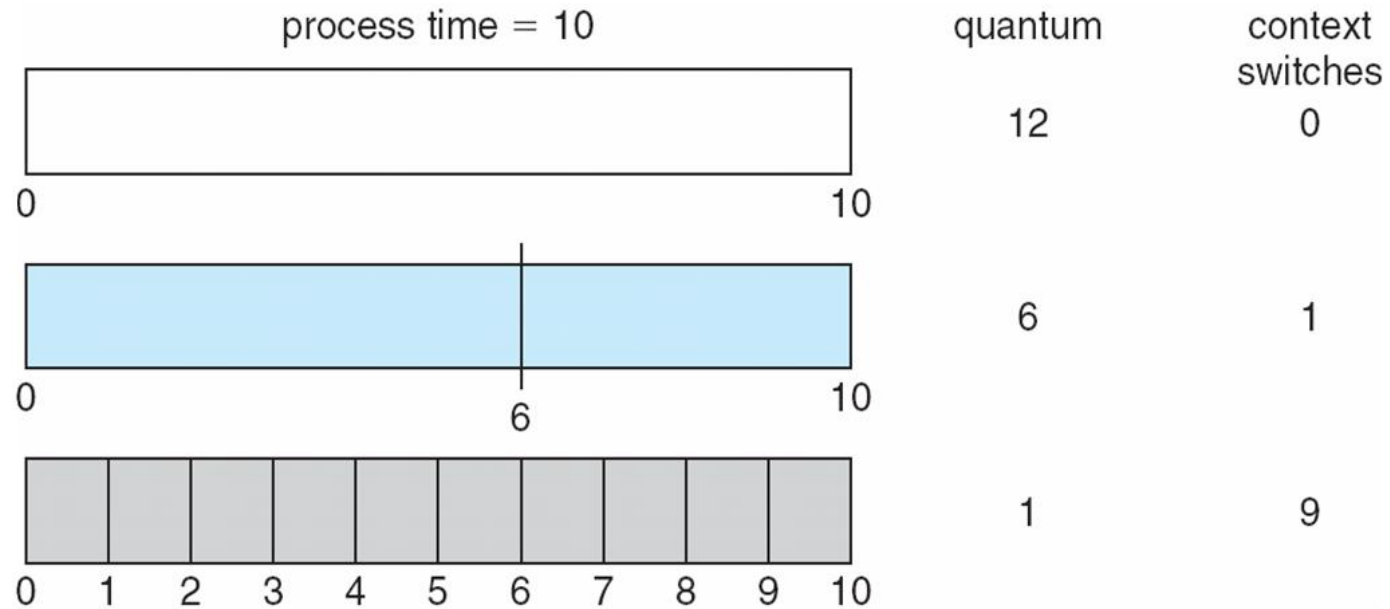


- Typically, higher average turnaround than SJF, but better **response**
- q should be large compared to context switch time
- q usually 10ms to 100ms, context switch < 10 usec



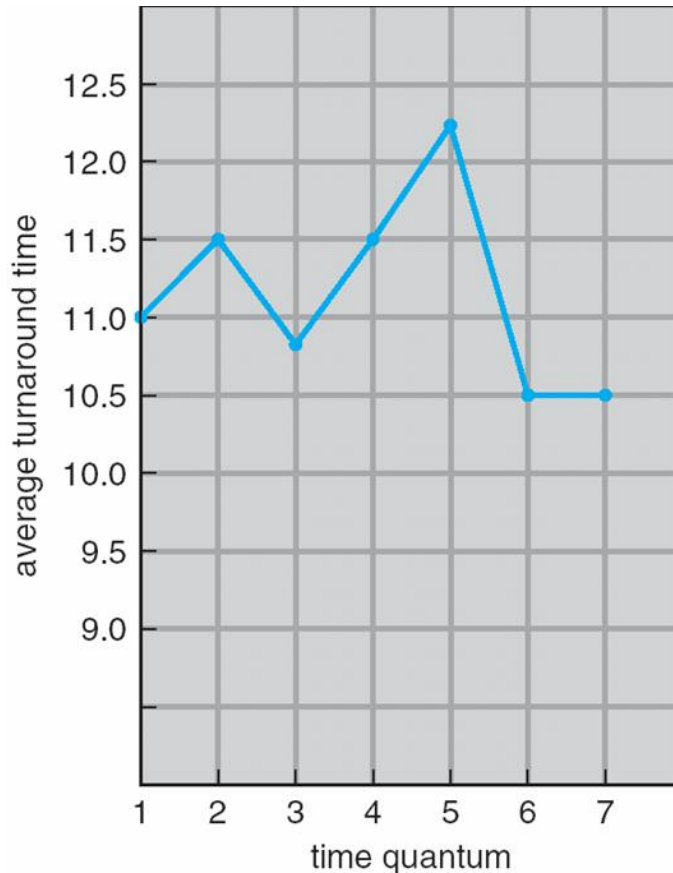


Time Quantum and Context Switch Time





Turnaround Time Varies With The Time Quantum



process	time
P_1	6
P_2	3
P_3	1
P_4	7

80% of CPU bursts
should be shorter than q





Example

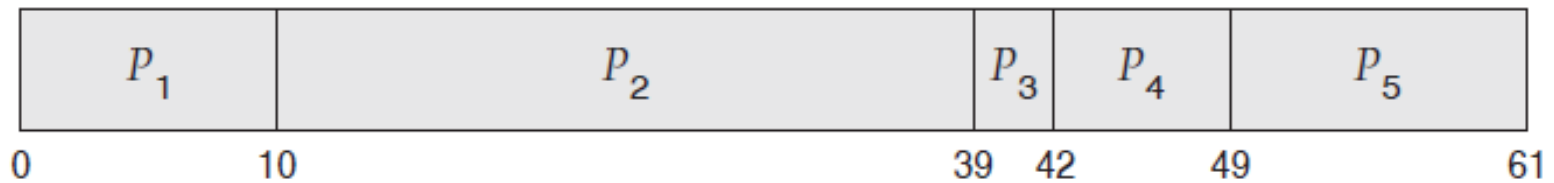
Consider the FCFS, SJF, and RR (quantum = 10 milliseconds) scheduling algorithms for this set of processes. Which algorithm would give the minimum average waiting time?

<u>Process</u>	<u>Burst Time</u>
P_1	10
P_2	29
P_3	3
P_4	7
P_5	12

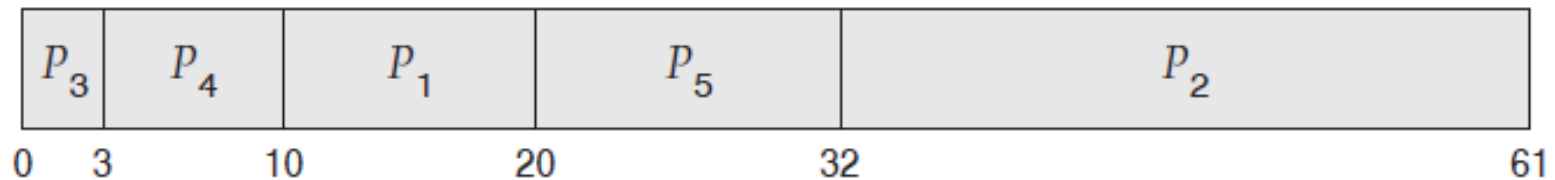




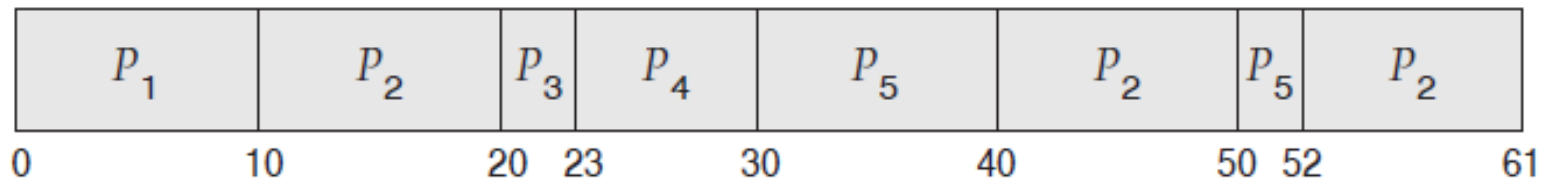
□ FCFS AWT=28ms



□ SJF AWT=13ms



□ RR AWT=23ms





Multilevel Queue

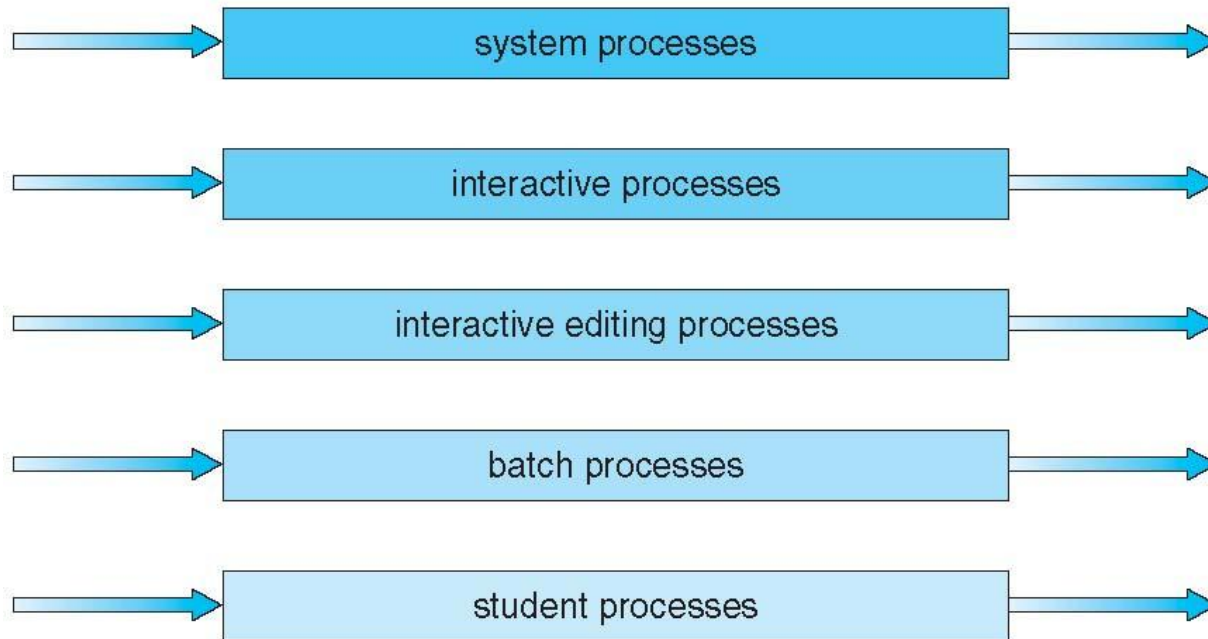
- processes are easily classified into different groups - **foreground** (interactive) processes and **background** (batch) processes.
- These have different response-time requirements and so may have different scheduling needs
- Ready queue is partitioned into separate queues, eg:
 - **foreground** (interactive)
 - **background** (batch)
- Process permanently in a given queue – based on memory size, process priority, or process type
- Each queue has its own scheduling algorithm:
 - foreground – RR
 - background – FCFS





Multilevel Queue Scheduling

highest priority



lowest priority

- ❑ Scheduling must be done between the queues:
- ❑ Fixed priority scheduling; (i.e., serve all from foreground then from background). Possibility of starvation.
- ❑ Time slice – each queue gets a certain amount of CPU time which it can schedule amongst its processes; i.e., 80% to foreground in RR
- ❑ 20% to background in FCFS





Multilevel Feedback Queue

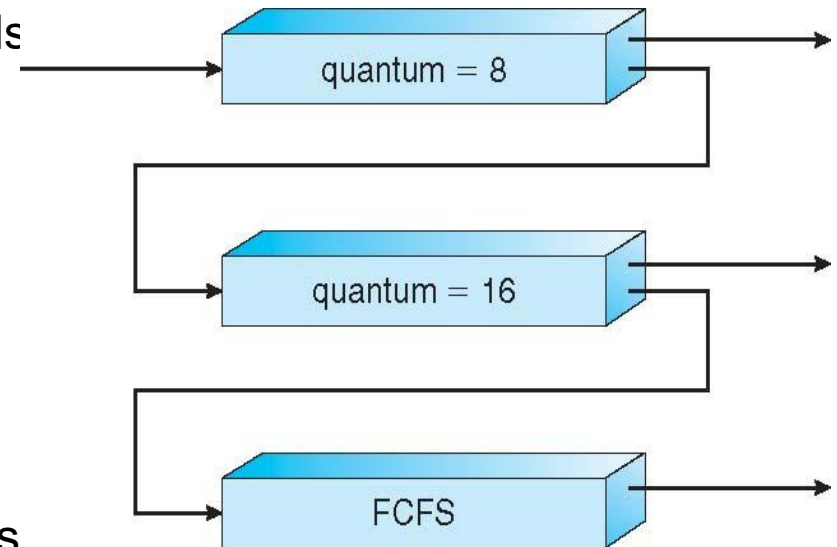
- ❑ A process can move between the various queues; aging can be implemented this way
- ❑ separate processes according to the characteristics of their CPU bursts.
- ❑ If a process uses too much CPU time, it will be moved to a lower-priority queue.
- ❑ leaves I/O-bound and interactive processes in the higher-priority queues.
- ❑ In addition, a process that waits too long in a lower-priority queue may be moved to a higher-priority queue - prevents starvation.
- ❑ Multilevel-feedback-queue scheduler defined by the following parameters:
 - ❑ number of queues
 - ❑ scheduling algorithms for each queue
 - ❑ method used to determine when to upgrade a process
 - ❑ method used to determine when to demote a process
 - ❑ method used to determine which queue a process will enter when that process needs service





Example of Multilevel Feedback Queue

- Three queues:
 - Q0 – RR with time quantum 8 milliseconds
 - Q1 – RR time quantum 16 milliseconds
 - Q2 – FCFS
- Scheduling
 - A new job enters queue Q0 which is served FCFS
 - When it gains CPU, job receives 8 milliseconds
 - If it does not finish in 8 milliseconds, job is moved to queue Q1
 - At Q1 job is again served FCFS and receives 16 additional milliseconds
 - If it still does not complete, it is preempted and moved to queue Q2





Thread Scheduling

- Distinction between user-level and kernel-level threads
- When threads supported, threads scheduled, not processes
- Many-to-one and many-to-many models, thread library schedules user-level threads to run on LWP(lightweight process)
 - Known as **process-contention scope (PCS)** since scheduling competition for the CPU takes place among threads belonging to the same process
 - When we say the thread library *schedules* user threads onto available LWPs, we do not mean that the thread is actually running on a CPU;
 - this would require the operating system to schedule the kernel thread onto a physical CPU.
 - Typically done via priority set by programmer
- To decide which kernel thread to schedule onto a CPU, the kernel uses **system-contention scope (SCS)**
- Competition for the CPU with SCS scheduling takes place among all threads in the system



End of Chapter 5

